Brain-to text: Decode intracortical neural activity during attempted speech into words

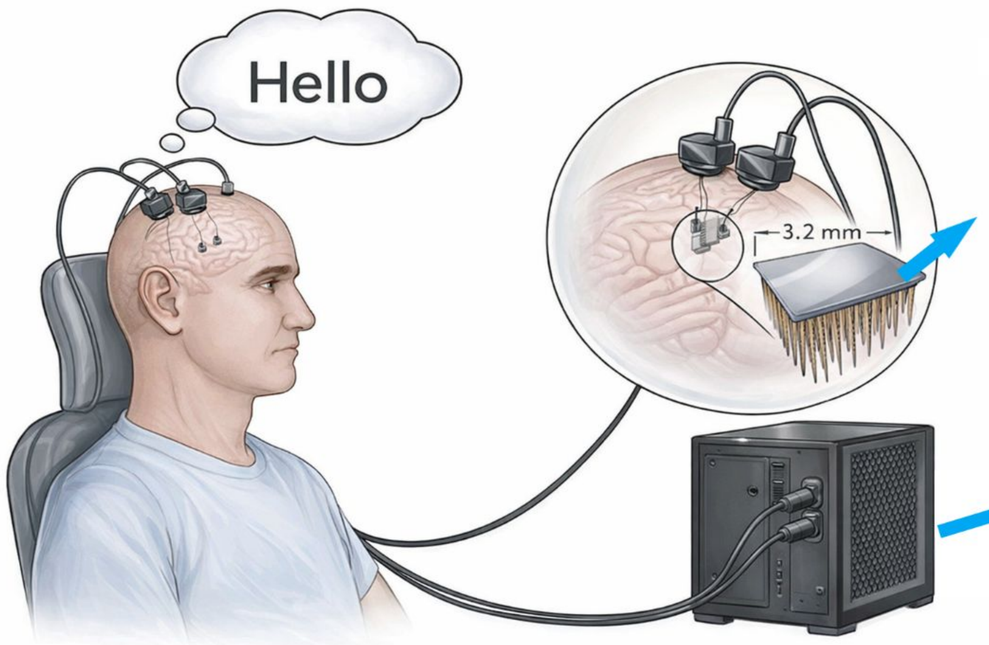
Rafael Nahon, Sabir Jacquir
Université Paris-Saclay, CNRS, Institut des Neurosciences Paris-Saclay (NeuroPSI),
91400 Saclay, France

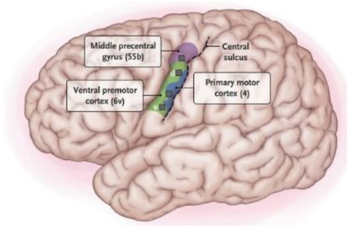
Introduction

The Brain-to-Text 2025 dataset is derived from intracortical speech neuroprosthesis experiments (Card et al., NEJM), where a participant attempted speech during a copy task and neural activity was decoded into phonemes and then words. The training data spans **44 recording sessions**, each containing roughly **100–300 trials**. The competition's end goal is to minimize **WER (Word Error Rate)** with an algorithm that is accurate but also fast enough to be practical. In our work, we focused on **PER (Phoneme Error Rate)** instead of WER because computing WER requires an additional phoneme-to-word decoding stage that can be slow and resource-intensive, while PER is strongly linked to downstream word accuracy. Improving PER is also a relatively under-explored direction compared to heavy language-model decoding. We trained multiple Transformer-CTC decoders and combined them with a lightweight voting-based ensemble, which reduced PER compared to the lab baseline while remaining extremely fast to run, suggesting strong potential for further gains with smarter voting strategies.

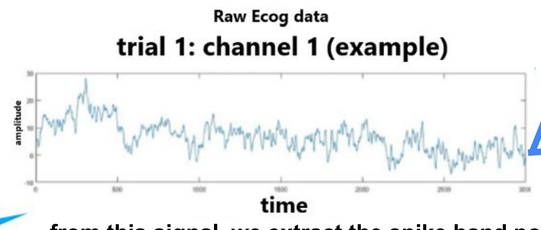
Material & Methods

Recording setup:

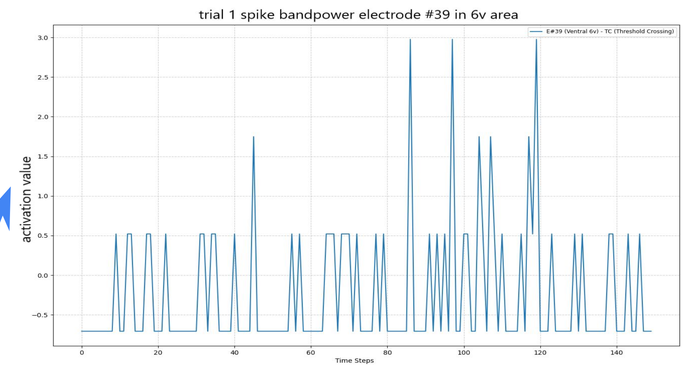


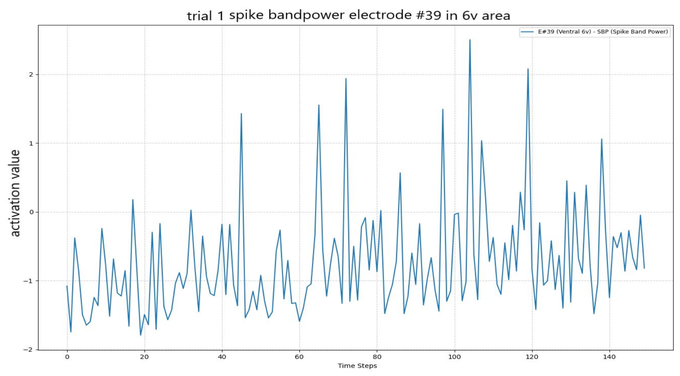


there was 4 arrays of 64 electrodes implanted:
area 6v: Two arrays, totaling 128 electrodes.
area 4: One array, totaling 64 electrodes.
Area 55b : One array, totaling 64 electrodes.



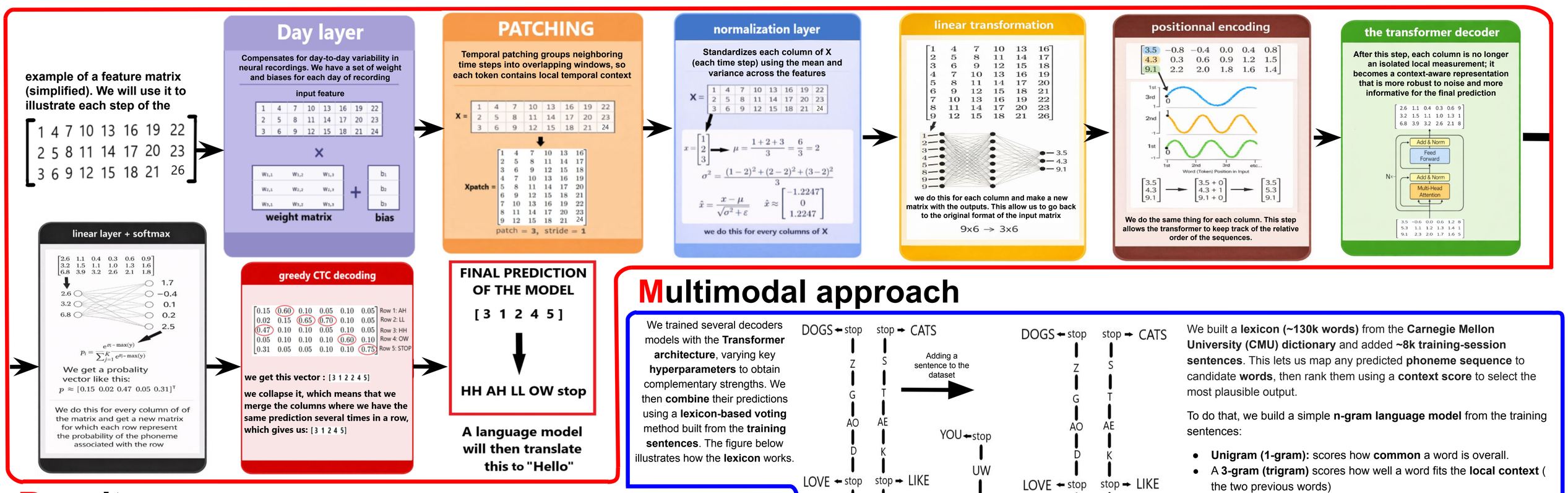
from this signal, we extract the spike band power (between 250 and 5000 Hz) and the threshold crossings (- 4.5 x RMS). They chose these features because they are simple, robust ways to summarize "how much neurons are firing" on each electrode, without needing to identify each neuron and its activity one by one.





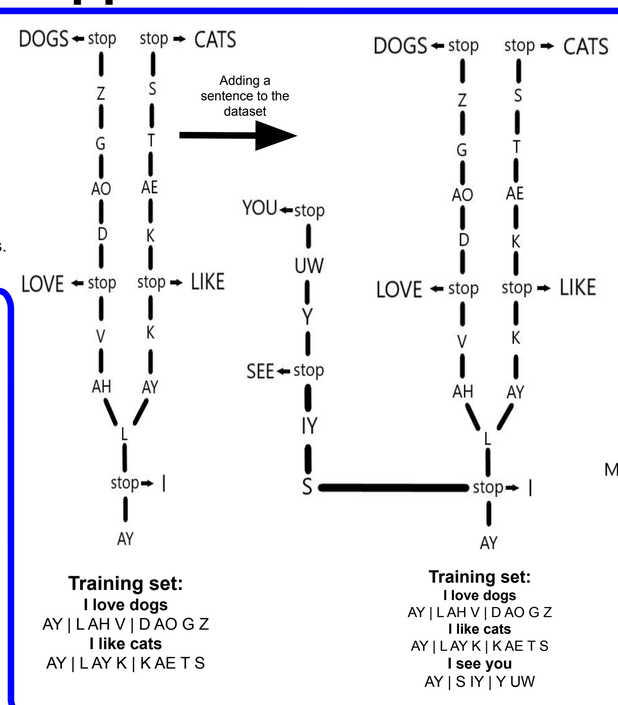
we have 256 channels, and for each channel we take 2 feature therefore, we get a matrix with 512 feature for each timestep.

Model architecture



Multimodal approach

We trained several decoders models with the **Transformer architecture**, varying key **hyperparameters** to obtain complementary strengths. We then **combine** their predictions using a **lexicon-based voting** method built from the **training sentences**. The figure below illustrates how the **lexicon** works.

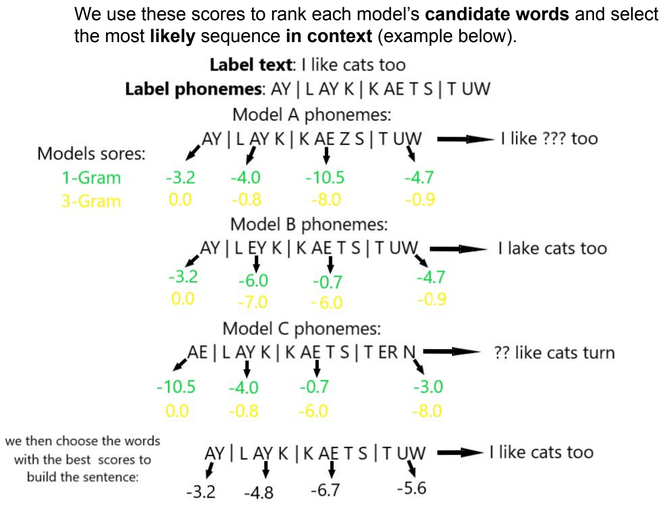


We built a **lexicon (~130k words)** from the **Carnegie Mellon University (CMU) dictionary** and added **~8k training-session sentences**. This lets us map any predicted **phoneme sequence** to candidate **words**, then rank them using a **context score** to select the most plausible output.

To do that, we build a simple **n-gram language model** from the training sentences:

- **Unigram (1-gram):** scores how **common** a word is overall.
- A **3-gram (trigram)** scores how well a word fits the **local context** (the two previous words)

We use these scores to rank each model's **candidate words** and select the most **likely** sequence **in context** (example below).



Results

We trained five **Transformer-CTC decoders** with the same overall design but different **model sizes** (number of trainable **parameters**: weights + biases). Larger models usually have higher **capacity** to learn patterns and context from neural activity and map them to **phoneme sequences**.

- **Model 1 — Day-aware + patching (≈50M parameters):** 10-layer Transformer (512 hidden units), with patching (patch size 14, stride 4) and a day-aware input dropout to improve robustness across recording sessions.
- **Model 2 — Day-aware + patching (≈45M parameters):** same setup as Model 1, but with a reduced feed-forward capacity (fewer decoder weights).
- **Model 3 — Day-aware + patching (≈40M parameters):** same setup as Model 2, with an additional reduction in decoder capacity (smaller feed-forward block).
- **Model 4 — Standard Transformer-CTC (≈30M parameters):** 10-layer Transformer (512 hidden units, 8 heads), trained **without** patching and **without** the day-aware component.
- **Model 5 — standard Transformer-CTC (≈20M parameters):** 11-layer Transformer, narrower (384 hidden units, 8 heads)

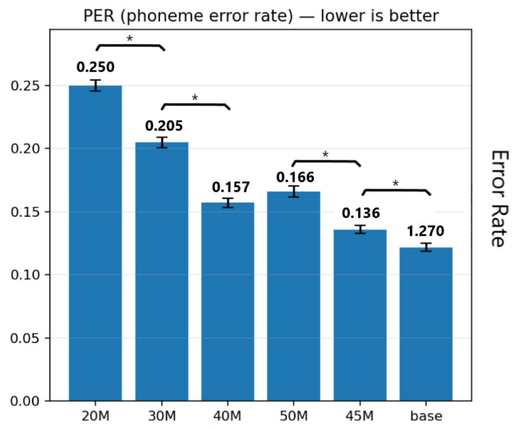


Figure 1: training results

Figure 1 reports **phoneme error rate (PER)** for all trained models, and also includes the **baseline model** provided by the lab that collected the data. The results show that removing the **day layer** and **temporal patching** leads to **significantly worse performance**. Among the three models that include **day layer + patching**, the **45M-parameter** model achieves the **best PER** and comes **closest to the baseline**, although the **baseline** still performs better overall.

We then used a **multimodel ensemble** to test whether combining decoders could improve accuracy. With **6 models**, there are **57 possible subsets** in total, but this heatmap shows only the **pairwise (2-model) combinations** (off-diagonal), with each model's **individual PER** on the diagonal.

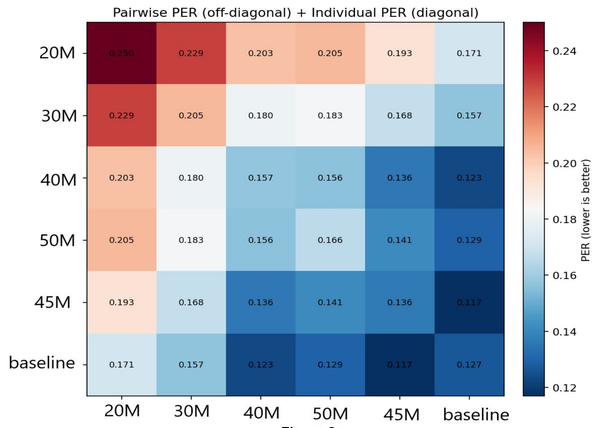


Figure 2:

This heatmap reports **PER (lower is better)** for our six systems. The **diagonal** shows each model's PER alone, while **off-diagonal** cells show the PER obtained b **combining two models**. Overall, ensembling tends to reduce PER, with the best results achieved by pairing the strongest models: **45M + baseline** yields a **significant** PER improvement over the baseline alone (**t-test**).

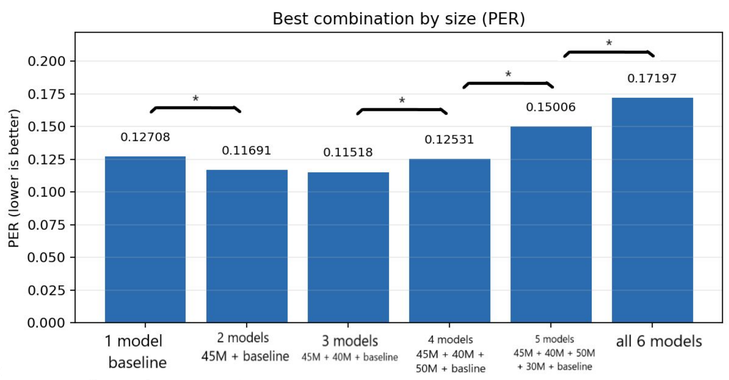


figure 3: Best-performing combinaisons by size. Each bar shows the **best ensemble** for a given number of models (**1–6**) and the **models included**. The performance drop beyond **2–3 models** likely show that weaker models with higher **individual PER** start to **dilute the vote** or reflects **diminishing returns** from adding more decoders.

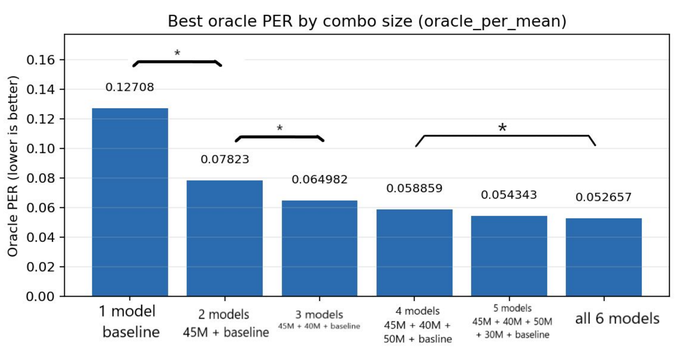


Figure 4: Best oracle PER by ensemble size (1–6). We report the **oracle prediction** for the best model combinations at each size.

Oracle = an upper bound that selects the best option *after seeing the reference*. In the **word-oracle**, at each word position we compare each model's predicted **phoneme sequence** to the **ground-truth** word and keep the closest one. Example: target is **"the code"**; model 1 predicts **"the cold"** and model 2 predicts **"the code"**. An **n-gram vote** might choose "the **cold**", but the **oracle** keeps **"the code"**.because it matches the label more closely.

This is **not deployable**, it's a **best-case benchmark** that helps estimate the **potential PER** achievable by these model combinations if we design a stronger voting method.

Conclusion

We trained decoders with performance close to the lab **baseline**, and further reduced **PER** by combining multiple models with a **voting-based ensemble**. Although some models are less accurate on average, the **oracle analysis** shows that adding models can lower the best achievable (**oracle**) **PER**: weaker models may still predict words that stronger models miss. While the **oracle** is an unattainable upper bound, more advanced **voting strategies** could help close this gap. Importantly, the method is **very fast**: with **6 models**, inference plus voting takes **< 0.5 s**, leaving room for additional lightweight **post-processing** to move closer to oracle performance.

In this poster, we focus on **PER** rather than **WER**, because reliable WER evaluation requires converting **phonemes → words** with **language models**, which can be **computationally expensive**. By contrast, our ensemble operates directly at the **phoneme level**, so it remains **fast** and **resource-efficient** while delivering clear accuracy gains. As **PER** decreases, phoneme sequences become less ambiguous, enabling **simpler and cheaper phoneme-to-word decoding** (example: small **n-gram models**) instead of heavy language modeling.

A key limitation is our **dataset and evaluation protocol**: no session was reserved for **true unseen-session testing**, and each recording day contains only a small number of trials with a limited within-session split. As a result, **day-aware models** are mainly optimized for sessions seen during training. We also could not fairly evaluate the lab **baseline** on fully unseen sessions because it was trained across **all sessions**.

To partially address this, we held out one entire session when training our Transformer models and observed an absolute **~3% PER degradation**, indicating a **generalization gap** that would matter in real-world use. This gap could be reduced by initializing from the **closest session's day weights** and applying lightweight **online adaptation** to calibrate the model to a new recording session.

Références:

Card NS, Wairagkar M, Iacobacci C, et al. **An Accurate and Rapidly Calibrating Speech Neuroprosthesis.** *N Engl J Med.* 2024;391(7):609–618. doi:10.1056/NEJMoa2314132.